

Topic 11: Heaps and Priority Queues

11.1 Heap Properties

- A complete binary tree stored efficiently as an array
- Max-Heap: every parent $\geq$ its children -- root is the maximum element
- Min-Heap: every parent $\leq$ its children -- root is the minimum element
- For node at index i : left child at $2i+1$, right child at $2i+2$, parent at $(i-1)/2$

11.2 Heap Operations

- Insert (push): add element at the end, then sift up (swap with parent while parent smaller)
- Extract-Max/Min (pop): swap root with last element, reduce size, sift down (swap with larger child)
- Both insert and extract are $O(\log n)$
- Build heap from unsorted array: $O(n)$ -- start sifting down from last internal node

```
class MaxHeap {
    vector<int> h;
    void siftUp(int i) {
        while (i>0 && h[(i-1)/2] < h[i]) {
            swap(h[i], h[(i-1)/2]); i = (i-1)/2;
        }
    }
    void siftDown(int i) {
        int n = h.size();
        while (2*i+1 < n) {
            int big = 2*i+1;
            if (2*i+2 < n && h[2*i+2] > h[big]) big = 2*i+2;
            if (h[i] >= h[big]) break;
            swap(h[i], h[big]); i = big;
        }
    }
public:
    void push(int v) { h.push_back(v); siftUp(h.size()-1); }
    int top() { return h[0]; }
    void pop() { h[0]=h.back(); h.pop_back(); siftDown(0); }
};
```

11.3 Heap Sort

- Build a max-heap from the array in $O(n)$
- Repeatedly extract the max and place at the end: $O(n \log n)$ total

- In-place and $O(n \log n)$ worst-case -- but poor cache performance compared to quicksort

11.4 STL Priority Queue

- `std::priority_queue<int>`: max-heap
 - `push()`, `pop()`, `top()`, `empty()`, `size()`
 - Custom comparator for min-heap: `priority_queue<int, vector<int>, greater<int>>`
 - Applications: Dijkstra's algorithm, A* search, task scheduling, K largest elements
-